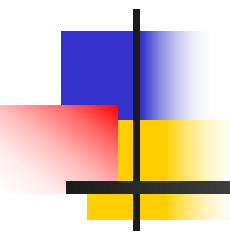


C语言与程序设计

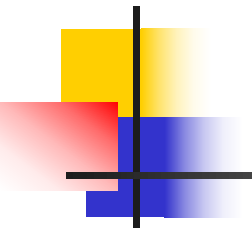
The C Programming Language



第5章 函数

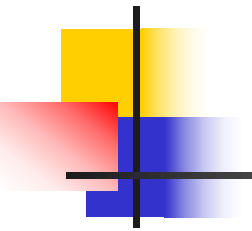
华中科技大学计算机学院

卢萍



主要内容

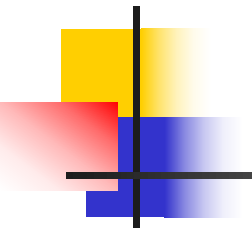
- 结构化编程和C程序的一般结构
- 函数的机制：函数定义、函数声明、函数调用
- 变量的存储类型
- 递归



5.1 模块化程序设计

把一个问题逐步细化分解为若干子问题，用函数实现子问题。

- 使程序编制方便，易于管理、修改和调试。
- 增强了程序的可读性、可维护性和可扩充性，方便于多人分工合作完成程序的编制。
- 函数可以公用，避免在程序中使用重复的代码。
- 提高软件的可重用性，软件的可重用性是转向面向对象程序设计的重要因素。



蒙特卡罗模拟：猜数游戏

计算机随机产生一个数（1~1000），游戏者猜直到正确为止。

- **模拟算法：**编程实现现实世界中的随机事件，例如，抛硬币、掷骰子和玩牌等。
- **蒙特卡罗模拟：**使用随机数来模拟。
- **随机数：**具有不确定性和偶然性特点，应用领域：
软件测试——测试数据，加密系统——密钥，网络——验证码
- **随机数发生器：**生成随机数的函数

```
int rand (void) ;    // 在 stdlib.h中
```
- **伪随机数：**依靠计算机内部算法产生的“随机”数

主程序结构

do {

计算机产生一个1到1000的随机数;

游戏者猜数, 直至猜对;

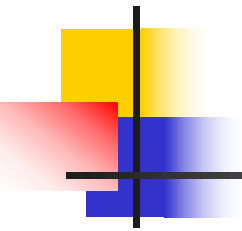
printf("Play again? (Y/N) ");

scanf("%1s", &cmd);

} while (cmd == 'y' || cmd == 'Y');

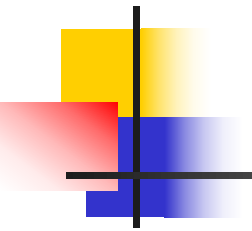
void GuessNum(int x);

int GetNum(void);



主程序结构

```
do {  
    magic = GetNum( ); /* 产生随机数*/  
    GuessNum(magic);  /* 猜数 */  
    printf("Play again? (Y/N) ");  
    scanf("%1s", &cmd);  
} while (cmd == 'y' || cmd == 'Y');
```



子任务1: GetNum(void)

- ① 调用标准库函数rand产生一个随机数;
- ② 将这个随机数限制在1~1000之间。

利用函数，可以实现程序的模块化，把程序中常用的一些算法或操作编成通用的函数，以供随时调用，大大简化主函数的流程，使程序设计简单和直观，提高程序的易读性和可维护性。

int GetNum(void) ;

/******

函数名称: **GetNum**

函数功能: 产生一个1到**MAX_NUMBER**之间的随机数, 供游戏者猜测。

函数参数: 无

函数返回值: 返回产生的随机数

*****/

```
int GetNum(void)                                /* 注意: 后面无分号 */
{
    int x;
    printf("A magic number between 1 and %d has been chosen.\n",
           MAX_NUMBER);
    x=rand();                                    /* 调用标准库函数rand产生一个随机数 */
    x= x % MAX_NUMBER + 1;                      /* 将这个随机数限制在1~MAX_NUMBER之间 */
    return(x);
}
```

rand是接口**stdlib.h**中的一个函数, 它返回一个 $\leq \text{RAND_MAX}$ 的正数, **RAND_MAX**的值取决于系统。
windows: 32767 ($2^{15}-1$)
Linux: 2147483647 ($2^{31}-1$)

函数GuessNum

```
void GuessNum(int x)
{
    int guess, counter = 0;
    for (;;)
    {
        printf("guess it: ");
        scanf("%d", &guess);
        counter++;           /* 统计猜的次数 */
        if (guess == x)
        {                     /* 猜对 */
            printf("You guessed the number by %d times!\n\n", counter);
            return;          // break; 行吗
        }
        else if (guess < x) /* 猜小了 */
        {
            printf("Too low. Try again.\n");
        }
        else                /* 猜大了 */
        {
            printf("Too high. Try again.\n");
        }
    }
}
```

main函数

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_NUMBER 1000
int GetNum (void);      /* 函数原型 */
void GuessNum(int x);   /* 函数原型 */
int main(void)
{
    char command;
    int magic;
    do
    {
        magic = GetNum( ); /*调用GetNum产生随机数*/
        GuessNum(magic);   /* 调用GuessNum猜数 */
        printf("Play again? (Y/N) ");
        scanf("%1s", &command); /* 询问是否继续 */
    } while (command == 'y' || command == 'Y');
    return 0;
}
```



伪随机数算法

- 伪随机数是指用数学递推公式所产生的随机数。
- 应用最广的递推公式：（即线性同余法）

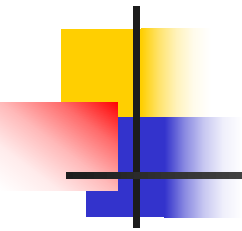
$$a_0 = \text{seed}$$

$$a_n = (A * a_{n-1} + B) \% M, n \geq 1$$

其中 A, B, M 是产生器设定的常数, 用户不能更改。

seed: 种子参数, 该发生器从称为种子（一个无符号整型数）的初始值开始用确定的算法产生随机数。

- ◆ 产生器给定了初始值
- ◆ **seed**应该在哪儿定义并初始化？
- ◆ 允许用户设置初始值（修改）
- ◆ 怎么修改？

- 
- 该发生器从称为**种子**（一个无符号整型数）的初始值开始用确定的算法产生随机数。显然，通过种子产生第一个随机数后，后续的随机序列也就是确定的了。由此可见，随机数的产生依赖于种子，为了使程序在反复运行时能产生不同的随机数，必须改变这个种子的值，这称为初始化随机数发生器，由**函数srand**来实现。

```
int main(void)
```

```
{
```

```
    char command;
```

```
    int magic;
```

```
    printf("This is a guessing game\n\n");
```

```
    srand(time(NULL)); /*用系统时间初始化随机数生成器 */
```

```
    do
```

```
    {
```

```
        magic = GetNum( ); /*调用GetNum产生随机数供猜测*/
```

```
        GuessNum(magic); /* 调用GuessNum猜出这个数 */
```

```
        printf("Play again? (Y/N) ");
```

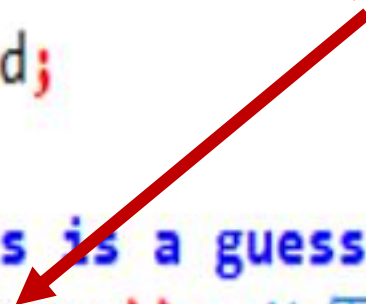
```
        scanf("%1s", &command); /* 问是否继续 */
```

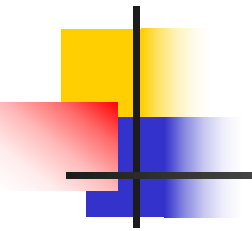
```
    } while (command == 'y' || command == 'Y');
```

```
    return 0;
```

```
}
```

初始化语句的位置

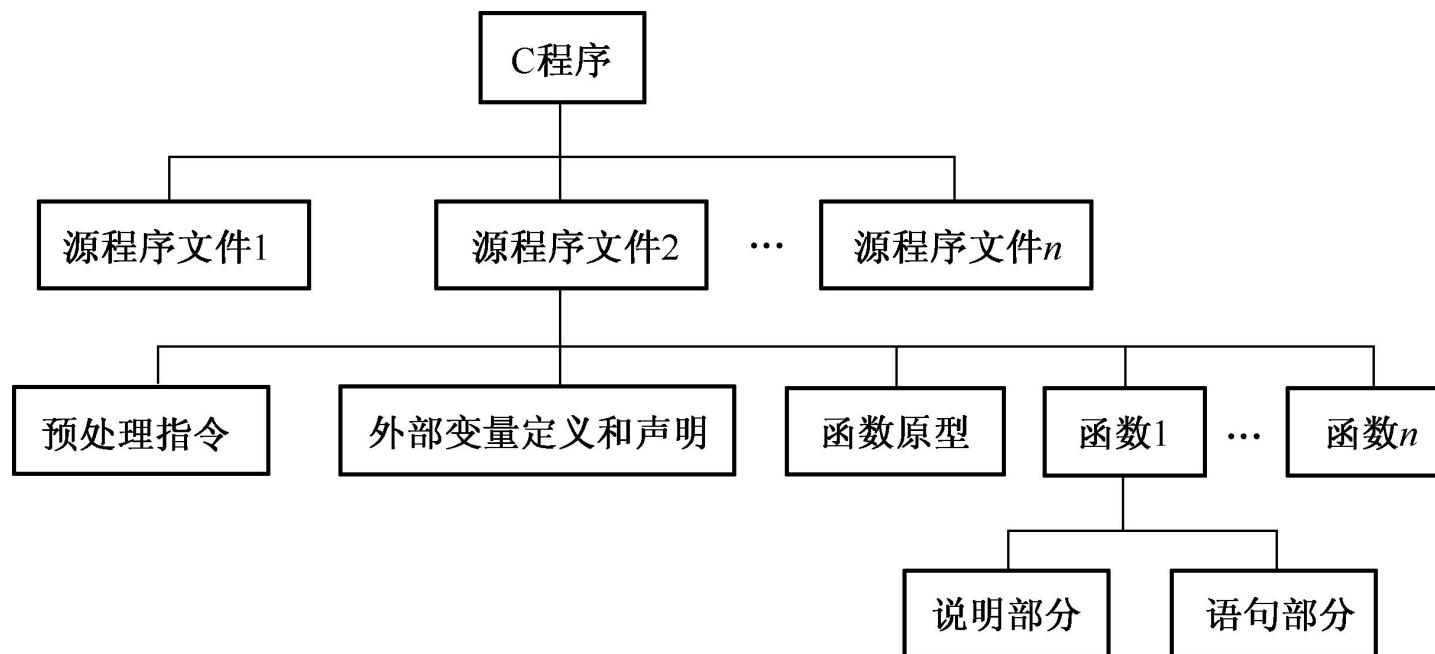




库函数srand

- **srand**函数是随机数发生器的初始化函数，原型为：
void srand (unsigned int x) ; //原型在**stdlib.h**中
// 用**x**作为种子，**rand**根据这个种子的值产生一系列随机数
- 用什么值做种子？
用系统时间（由**time**函数得到）作为种子。
函数**time**返回自**1970年1月1日**以来经历的秒数。
srand(time(NULL)); //函数**time**的原型在**time.h**中

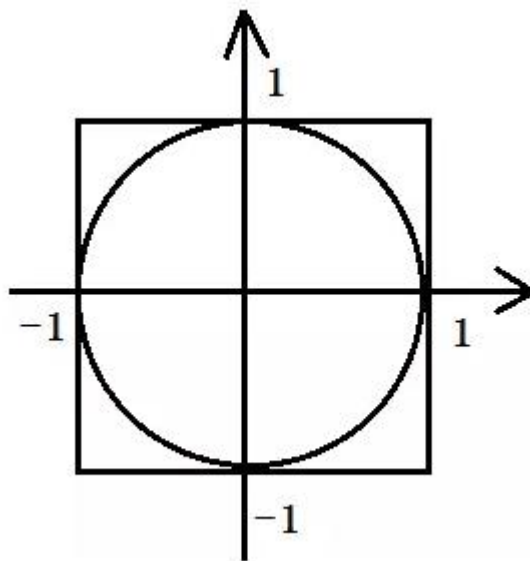
C程序的结构

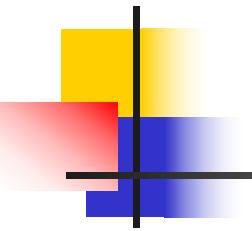


- 每个源文件可单独编译生成目标文件，组成一个**C**程序的所有源文件都被编译之后，由连接程序将各目标文件中的目标函数和系统标准函数库的函数装配成一个可执行**C**程序。

练习----蒙特卡罗法求圆周率近似值

- 有一个如图所示正方形及其内切圆，往正方形内扔飞镖 n 次，当 n 足够大，比值“落在圆内的次数/落在正方形内的次数 n ”将无限接近“圆的面积/正方形的面积”，即 $\pi/4$ 。 n 越大， π 的近似结果越精确。





5.2 自定义函数

程序中若要使用自定义函数实现所需的功能，需要做三件事：

- ① 按语法规则编写完成指定任务的函数，即定义函数；
- ② 有些情况下在调用函数之前要进行函数声明；
- ③ 在需要使用函数时调用函数

函数的定义

函数定义的一般形式为：

形式参数（简称形参）
无参数：**void**或空

类型名 函数名（参数列表）

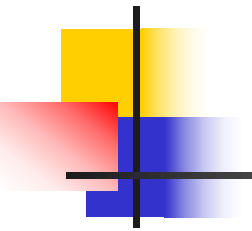
{

声明部分

语句部分

}

无返回值：类型为**void**



函数的返回值

- **return**语句可以是如下两种形式之一：
 - (1) **return ;** /* **void**函数：无返回值函数 */
 - (2) **return 表达式 ;** /* 非**void**函数：有返回值函数 */
- 一旦某个**return**语句被执行，控制立即返回到调用处，其后的代码不可能被执行。
- **void**函数也可以不包含**return**语句。如果没有**return**语句，当执行到函数结束的右花括号时，控制返回到调用处，把这种情况称为**离开结束**。
- 表达式值的类型应该与函数定义的返回值类型一致，如果不相同，将把表达式值的类型转换为函数的类型。

函数原型

- 函数定义出现在函数调用后
 - 被调用函数在其它文件中定义
- 必须在函数调用之前给出函数原型

类型名 函数名（参数类型表）；

`int max( int x , int y ) ;`

或

`int max( int , int ) ;`

无参数：必须写void

```
int GetNum (void);      /* 函数原型 */
void GuessNum(int x)    /* 函数定义 */
{
    .....
}
```

```
int main(void)
{
    .....
    do
    {
        magic = GetNum( );    /* 函数调用 */
        GuessNum(magic);      /* 函数调用 */
        .....
    } while (command == 'y' || command == 'Y');
    return 0;
}
```

```
int GetNum( )    /* 函数定义 */
{
    .....
}
```

```
void GuessNum(int x)    /* 函数定义 */
```

```
{
```

```
.....
```

```
}
```

```
int main(void)
```

```
{ .....
```

```
    int GetNum (void);    /* 函数原型 */
```

```
    void GuessNum(int) ; /* 函数原型 ， 也可以 void GuessNum(int x) ; */
```

```
do {
```

```
    magic = GetNum( );    /* 函数调用 */
```

```
    GuessNum(magic);    /* 函数调用 */
```

```
.....
```

```
    } while (command == 'y' || command == 'Y');
```

```
    return 0;
```

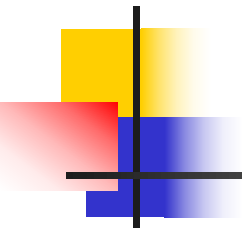
```
}
```

```
int GetNum( )    /* 函数定义 */
```

```
{
```

```
.....
```

```
}
```



良好的编程风格

- 在函数调用之前必须给出它的函数定义、函数原型或两者都给出。
- 引入标准头文件的主要原因是它含有函数原型。

函数调用与参数传递

- 函数调用的形式

函数名（实参列表）

实参是一个表达式
无参函数：为空

- 例如

`putchar(c);` // 作为表达式语句

`c=getchar();` // 作为表达式的操作数

`printf("%10.0f",sqrt(x));` // 作为实参

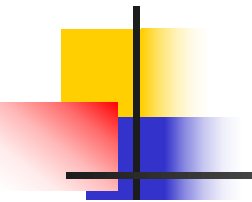
`GuessNum(GetNum());` // 作为实参

- 参数的传递方式是“值传递”，实参的值单向传递给相应的形参。如果实参、形参都是x，被调用函数不能改变实参x的值。

值传递示例

```
#include<stdio.h>
long power(int,int); /* 函数原型 */
int main(void)
{
    int x, n;
    for(x=1;x<=10;x++) { /* 分列显示整数1到10的2至5次幂 */
        for(n=2;n<=5;n++)
            printf("%10ld", power(x,n));
        printf("\n");
    }
    return 0;
}
long power(int x, int n) /* 计算x的n */
{
    long p;
    for (p=1;n>0;n--) p*=x;
    return p;
}
```

1	1	1	1
4	8	16	32
9	27	81	243
16	64	256	1024
25	125	625	3125
36	216	1296	7776
49	343	2401	16807
64	512	4096	32768
81	729	6561	59049
100	1000	10000	100000



实参的求值顺序

- 由具体实现确定，有的从左至右计算，有的按从右至左计算。

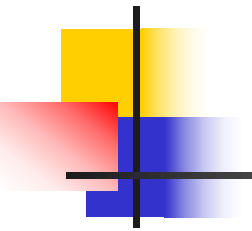
a=1;

power(a,a++)

----从左至右: **power(1,1)**

----从右至左: **power(2,1)** (多数)

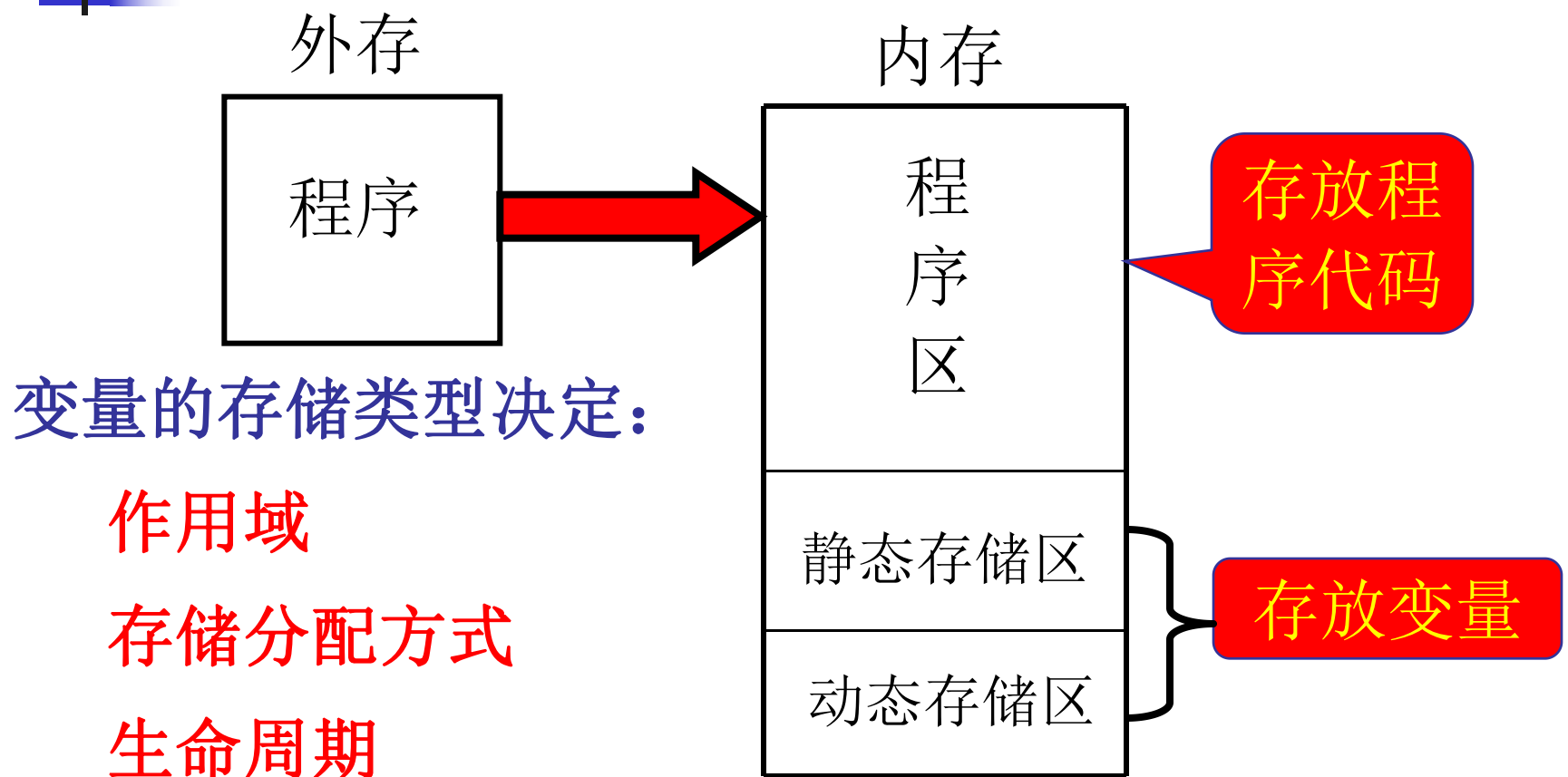
- 为了保证程序清晰、可移植，应避免使用会引起副作用的实参表达式。



传地址（引用）调用

- 将变量的地址值传递给函数，函数既可以使用，也可以改变该变量的值。标准库函数scanf就是一个引用调用的例子。
- `int x;`
`scanf("%d",&x);`

5.3 变量的存储类型



变量的存储类型决定：

作用域

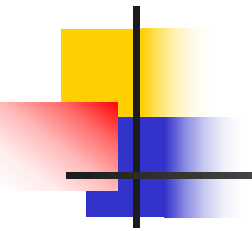
存储分配方式

生命周期

初始化方式

需要区分变量的存储类型

关键字有：auto、extern、static和register



作用域

- **作用域**是指标识符（变量或函数）的有效范围，也就是指程序正文中可以使用该标识符的那部分程序段。
- **可见性**：块多重嵌套时，同名外层变量不可见
- **按照作用域，变量分：**
局部变量和全局变量

局部变量和全局变量

- **局部变量**：在函数内部定义的变量，作用域是定义该变量的程序块，程序块是带有说明的复合语句（包括函数体）。不同函数可同名，同一函数内不同程序块可同名。**形式参数是局部变量**。
- **全局变量**：在函数外部定义的变量，其缺省作用域从其定义处开始一直到其所在文件的末尾，由程序中的部分或所有函数共享。

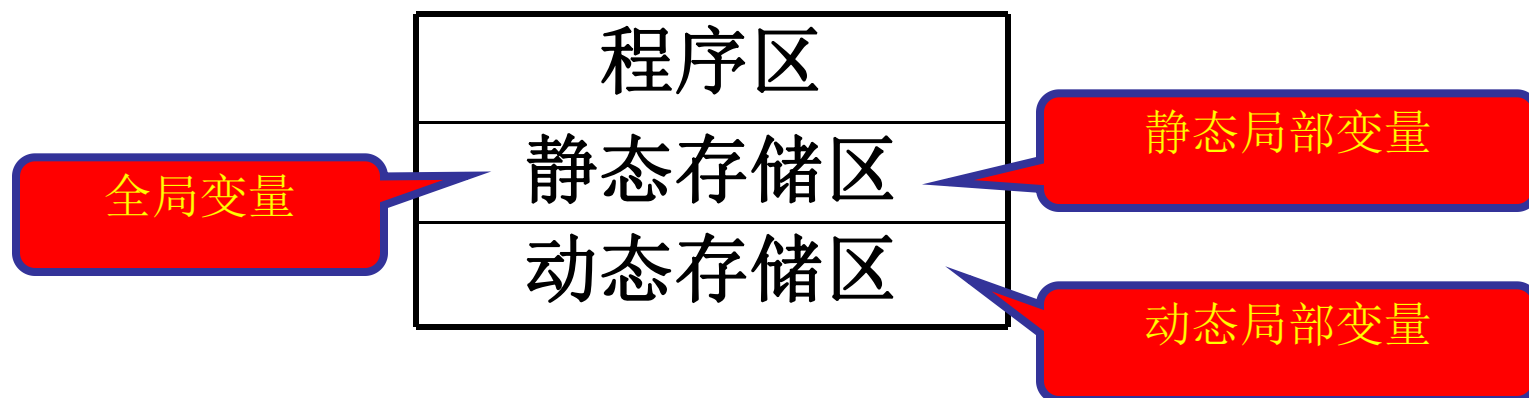
```
int p=1, q=5;    // p、q为全局变量
float f1( int a) // 形参a为局部变量
{ int b,c;       // b、c为局部变量
    ....
}
```

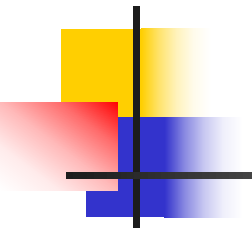
生存期

生存期 { 动态存储变量
静态存储变量

静态存储：在程序运行期间有固定的存储空间，直到程序运行结束。**全局变量属于静态存储**

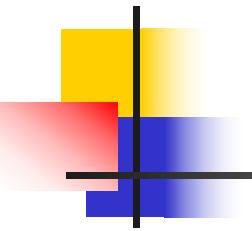
动态存储：在程序运行期间根据需要分配存储空间，所在块结束后立即释放空间。若一个函数在程序中被调用两次，则每次分配的单元有可能不同。**局部变量有动态和静态存储**





自动变量

- 局部变量的缺省存储类型是**auto**，称**自动变量**
- `auto int a; /*等价于int a; 也等价于auto a; */`
- **作用域**：局部于定义它的块，从块内定义之后直到该块结束有效。
- **存储分配方式**：动态分配方式,即在程序运行过程中分配和回收存储单元。
- **生命周期**：短暂的，只存在于该块的执行期间。
- **初始化方式**：定义时没有显示初始化，其初值是不确定的；有显示初始化，则每次进入块时都要执行一次赋初值操作。



外部变量

- 外部变量是全局变量，但定义时不用存储类关键字。
- 作用域:从定义之后直到该源文件结束的所有函数，通过用extern做声明，外部变量的作用域可以扩大到整个程序的所有文件。
- 存储分配方式:静态分配方式,即程序运行之前，系统就为外部变量在静态区分配存储单元，整个程序运行结束后所占用的存储单元才被收回。
- 生命周期: 永久的，存在于整个程序的执行期间。
- 初始化方式: 定义时没有显示初始化，初值0；有显示初始化，只执行一次赋初值操作。

extern声明

```
srand(int x)
```

```
{
```

```
    seed=x;
```

```
}
```

```
int seed=C; // seed作用域开始
```

```
int rand()
```

```
{
```

```
    seed = (A*seed+B) % M ;
```

```
    return seed;
```

```
}
```

引用出错：无作用
引用在前，定义在后

外部变量的定义
从定义点开始有效直至文件尾

extern声明

```
srand(int x)
```

```
{
```

```
    extern int seed;
```

```
    seed=x;
```

```
}
```

```
int seed=D;
```

```
int rand( )
```

```
{
```

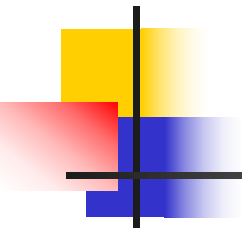
```
    seed = (A*seed+B) % M ;
```

```
    return seed;
```

```
}
```

外部变量的引用性声明语句

外部变量的定义性声明语句



定义性声明和引用性声明

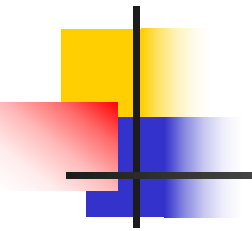
- ◆ 外部变量的定义性声明---- 分配存储单元
 - ✓ 位于所有的函数之外
 - ✓ 定义一次
 - ✓ 可初始化
- ◆ 外部变量的引用性声明----通报变量的类型
 - ✓ 位于在函数内或函数外
 - ✓ 出现多次
 - ✓ 不能初始化

外部变量 PK 形式参数

- 函数之间可以通过外部变量进行通信。
- 一般地，函数间通过形式参数进行通信更好。这样有助于提高函数的通用性，降低副作用。

```
int maxmin(int x,int y);  
extern int min,max;//引用性声明  
int main (void)  
{  
    maxmin (4 , 1) ;  
    printf("The max is %d\n",max);  
    printf("The min is %d\n",min);  
    return 0;  
}
```

```
int min, max;//定义性声明  
void maxmin (int x, int y)  
{  
    int m;  
    min=(x<y)?x : y;  
    max= (x>y)? x : y ;  
    return;  
}
```



静态变量

静态变量：关键字**static**修饰的变量

存储分配方式：静态分配方式

生命周期：永久的，

缺省初值：0,只执行一次

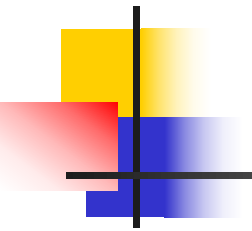
static有两个重要而截然不同的用法：

(1) 用于定义局部变量，称为静态局部变量。

(2) 用于定义外部变量，称为静态外部变量。

静态局部变量和自动变量有根本性的区别。

静态外部变量和外部变量区别：作用域不同。



静态局部变量

- 作用域：自动变量一样
- 编程计算 $1! + 2! + 3! + 4! + \dots + n!$
- 将求阶乘定义成函数，要求使用 **static** 使计算量最小。

静态局部变量

```
scanf("%d",&n);  
for(i=1;i<=n;i++)  
    sum+=fac(i);  
printf("1!+2!+...+%d!=%ld\n",n,sum);  
return 0;
```

```
}
```

```
/*  
*****
```

函数名称: fac

函数功能: 利用静态局部变量的特性求一个整数的阶乘。

函数参数: 形参n是int型

函数返回值: 返回n的阶乘, 类型long

```
*****  
*/
```

```
long fac(int n)
```

```
{
```

```
    static long f=1;    /* 静态局部变量 */
```

```
    f *=n;
```

```
    return f;
```

```
}
```


静态局部变量

```
srand(int x)
{
    seed=x;
}
```

seed在此无作用，
用户无法改变种子参数

```
int rand( )
{
    static int seed=D;
    seed = (A*seed+B) % M ;
    return seed;
}
```

静态局部变量
存于静态区，
初始化只执行1次

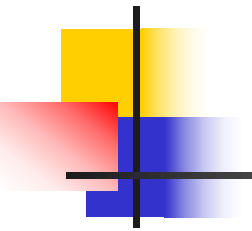
可以产生随机系列

静态外部变量

```
static int seed=D;
srand(int x)
{
    seed=x;
}
int rand( )
{
    seed = (A*seed+B) % M ;
    return seed;
}
```

静态外部变量

只能作用于定义它的文件，
其它文件中的函数不能使用



静态外部变量

- 静态外部变量和外部变量的区别是作用域的限制。
- 静态外部变量只能作用于定义它的文件，其它文件中的函数不能使用，
- 外部变量用**extern**声明后可以作用于其它文件。
- 使用静态外部变量的好处在于：当多人分别编写一个程序的不同文件时，可以按照需要命名变量而不必考虑是否会与其它文件中的变量同名，**保证文件的独立性**。
- 和局部变量能够屏蔽同名的外部变量一样，一个文件中的静态外部变量能够屏蔽其他文件中同名的外部变量。在静态外部变量所在的文件中，同名的外部变量不可见。

寄存器变量

- 寄存器变量：用**register**定义的局部变量
- **register****建议**编译器把该变量存储在计算机的高速硬件寄存器中，除此之外，其余特性和自动变量完全相同。
- 使用**register**的目的是为了提高程序的执行速度。程序中最频繁访问的变量，可声明为**register**。

```
register int i;          /* 等价于register i; */  
for (i=0;i<=N;i++) {    ...}
```

- 不可多，必要时使用。
- 无地址，不能使用**&**运算。

5.4 递归

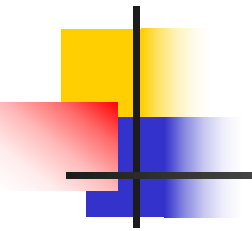
// 输出 n ~ 1

```
void prn_int(int n)
{
    if (n>0) {
        printf ("%d ",n);
        prn_int(n-1);
    }
}

int main(void)
{ prn_int(5);
  return 0;
}
```

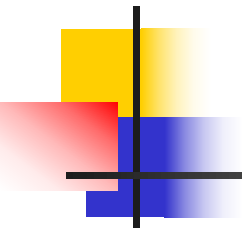
递归函数：
在定义中含有递归调用

递归调用：调用自己



递归概述

- 递归是一种函数在其定义中直接或间接调用自己的编程技巧。递归策略只需少量代码就可描述出解题过程所需要的多次重复计算，十分简单且易于理解。
- 递归调用：函数直接调用自己或通过另一函数间接调用自己的函数调用方式
$$f() \{ \dots f(); \dots \}$$
- 递归函数：在函数定义中含有递归调用的函数

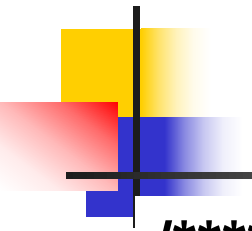


递归法求n!

- 阶乘的计算是一个典型的递归问题。**n!** 定义为:

$$n! = \begin{cases} 1 & n = 0, 1 \\ n \times (n-1)! & n > 1 \end{cases}$$

- 这是递归定义式，对于特定的**k**，**k!**只与**k**和**(k-1)!**有关，上式的第一式是递归结束条件，对于任意给定的**n!**，将最终求解到**1!** 或 **0!**。



$n!$ 的递归实现

```
/******
```

函数功能：递归法求一个整数的阶乘。

函数参数：参数 n 为int,表示要求阶乘的数。

函数返回值： n 的阶乘值，类型为long。

```
*****/
```

```
long fact(int n)
```

```
{
```

```
    if (n==0||n==1)        /* 递归结束条件 */
```

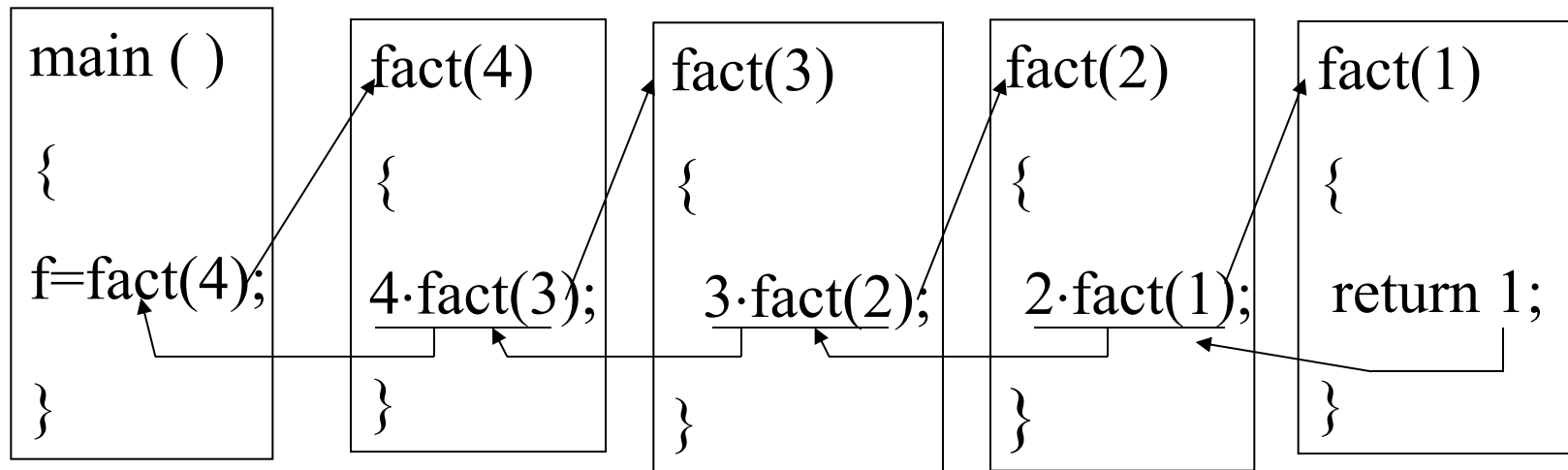
```
        return 1;
```

```
    else
```

```
        return (n*fact(n-1)); /* 递归调用 */
```

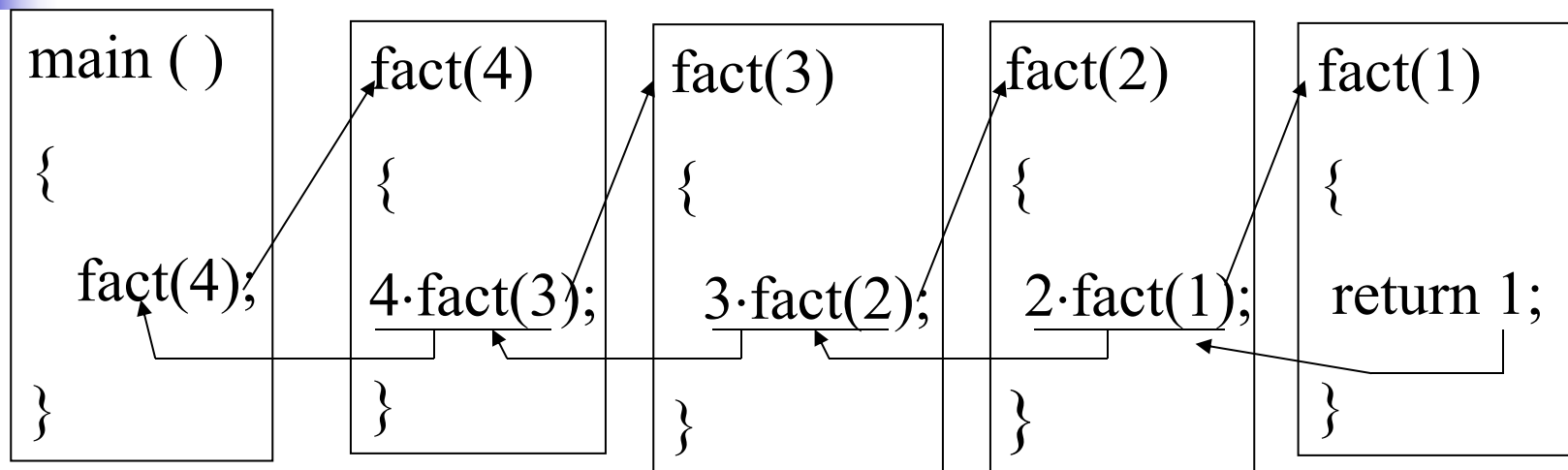
```
}
```


4! 的递归执行过程

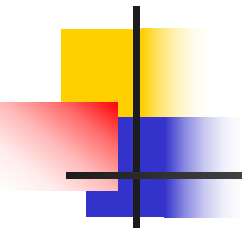


```
long fact(int n)
{
  if (n==0||n==1)      /* 递归结束条件 */
    return 1;
  else
    return (n*fact(n-1)); /* 递归调用 */
}
```

4! 的递归执行过程

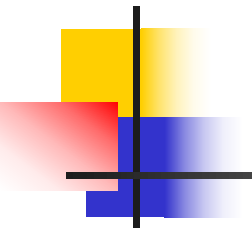


- ◆ 把求解问题转化为规模较小的子问题，通过多次递归一直到可以得出结果的最小解，然后通过最小解逐层向上返回调用，最终得到整个问题的解。
- ◆ 将递归概括为一句话：“能进则进，不进则退”
- ◆ 简化算法（易于理解）、但不节省存储空间，运行效率也不高(运行时开销大，效率低)



$n!$ 的迭代实现

```
/* 迭代法计算 $n!$  */  
long factorial_iteration( int n )  
{  
    int result = 1;  
    while( n>1 ) {  
        result *=n;  
        n--;  
    }  
    return result;  
}
```



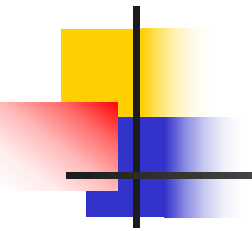
对比

◆ 两种方式的比较

两种实现方式都非常简单易懂，在实际项目中，为了效率，应该优先选择迭代。

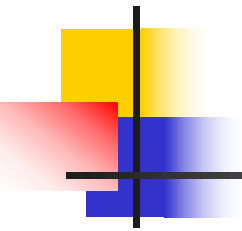
◆ 什么情况下使用递归

用递归能容易编写和维护代码，且运行效率并不至关重要。



递归算法的特点

- 递归算法的运行效率较低，耗费的计算时间较长，占用的存储空间也较多。
 - 递归算法结构紧凑、清晰、可读性强、代码简洁。
 - 大多数的简单递归函数都能改写为等价的迭代形式。
- 什么情况下使用递归呢？如果用递归能容易编写和维护代码，且运行效率并不至关重要，那么就使用递归。例如，像二叉树这样的数据结构，由于其本身固有的递归特性，特别适合于用递归处理；像回溯法等算法，一般也用递归来实现。

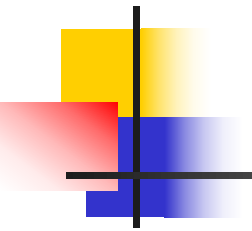


递归的要素

(1) 每次调用在规模上有所缩小。

(2) 递归结束条件

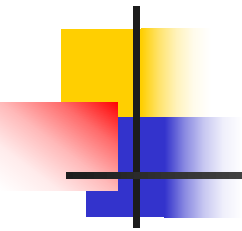
当子问题的规模足够小时，必须能够直接求出该规模问题的解。



递归求Fibonacci数列的第n项

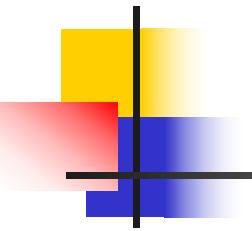
```
/* Recursive fibonacci : Compute the n item */  
long long fibonacci (long n )  
{  
    if ( n == 1 || n == 2) return 1;  
    else  
        return fibonacci(n-1)+fibonacci(n-2);  
}
```

这种递归方式有损于运行效率——如何改进？



递归函数设计

- 递归是一种强大的解决问题的技术，其基本思想是将复杂问题逐步转化为稍微简单一点的类似问题，最终将问题转化为可以直接得到结果的最简单问题。在较高级的程序中，递归是一个很重要的概念。



字符串的递归处理

- 字符串是以空字符（'\0'）结尾的字符序列
- 可以把字符串看成：一个字符后面再跟一个字符串或者空串
- 这是递归定义，可以用递归的方法对一些基本的字符串处理函数进行编写。

递归实现标准库函数strlen(s)



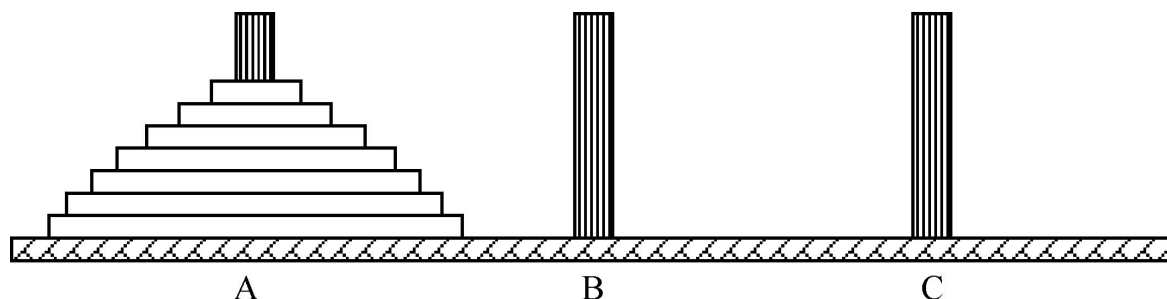
```
int strlen(char s[ ])
{
    if(s[0]=='\0')
        return 0;
    else
        return(1+strlen(s+1));
}
```

A red arrow points from the **s+1** in the recursive call to a red box containing the text **&s[1]**.

&s[1]

汉诺塔问题

- **问题：**木桩**A**上有**64**个盘子，盘子大小不等，大的在下，小的在上。把木桩**A**上的**64**个盘子都移到木桩**C**上，条件是一次只允许移动一个盘子，且不允许大盘放在小盘的上面，在移动过程中可以借助木桩**B**。



汉诺塔问题的递归算法

- 这是一个典型的用递归方法求解的问题。要移动 n 个盘子，可先考虑如何移动 $n-1$ 个盘子。分解为以下3个步骤：
 - (1) 把A上的 $n-1$ 个盘子借助C移到B。
 - (2) 把A上剩下的盘子（即最大的那个）移到C。
 - (3) 把B上的 $n-1$ 个盘子借助A移到C。
- 其中，第（1）步和第（3）步又可用同样的3步继续分解，依次分解下去，盘子数目 n 每次减少1，直至 n 为1结束。这显然是一个递归过程，递归结束条件是 n 为1。

函数move(n,a,b,c)

/ 将n个盘从a借助b，移至c */*

```
void move(int n,int a,int b,int c )
```

```
{
```

```
    if (n==1) printf(" %c-->%c\n ", a, c);
```

```
    else {
```

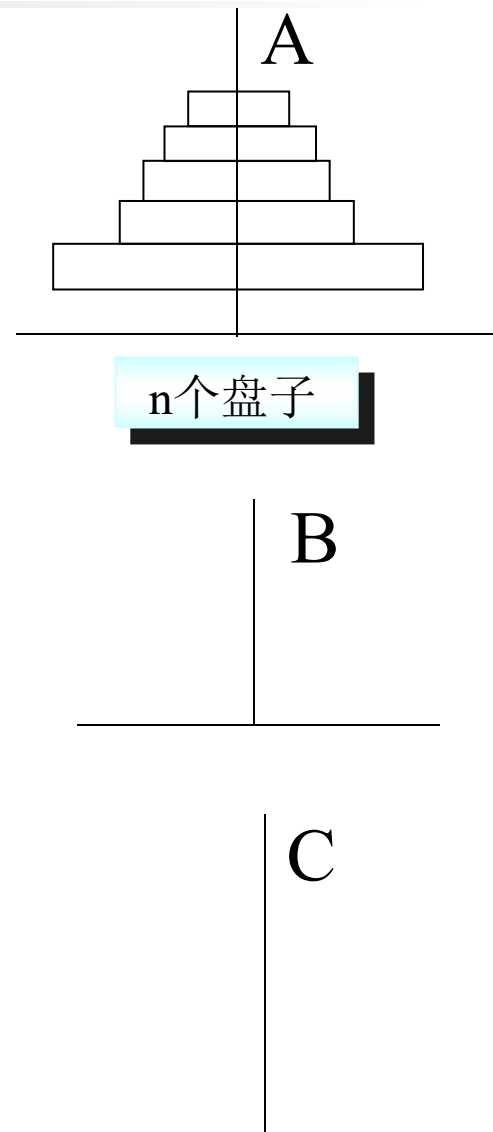
```
        move (n-1, a,c, b);
```

```
        printf(" %c-->%c\n ", a, c);
```

```
        move (n-1, b, a, c);
```

```
    }
```

```
}
```





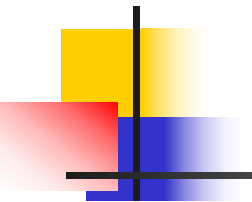
递归练习

1、用递归实现**reverse**函数：将输入的非负长整数逆序输出。如**reverse(123)**，则输出**321**。函数**reverse**的原型为：

```
void reverse(long m);
```

2、正反序相同的字符串被称为“回文字符串”。例如**LeveL**就是一个回文字符串。使用递归实现**is_palindrome**函数：判断一个串是否回文，是，返回**1**；不是，返回**0**。函数**is_palindrome**的原型为：

```
int is_palindrome(char str[ ],int n);
```



整数的分划问题

- ◆ 就是把正整数**M**写成一系列正整数之和的表达式。
注意，分划与顺序无关，即**5+1**和**1+5**是同一种分划。
另外，**M**本身也算是一种分划。例如，**M=6**时，可以分划为：

6

5+1

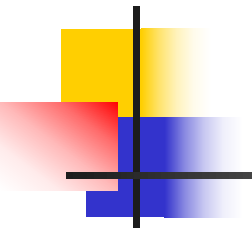
4+2, 4+1+1

3+3, 3+2+1, 3+1+1+1

2+2+2, 2+2+1+1, 2+1+1+1+1

1+1+1+1+1+1

- ◆ 输入正整数**m**，输出**m**的所有分划。



递归原则

1. 先设定分划 m 的第 0 位置的值 (i)，再从余下的 $m-i$ 设定下一个位置的值，...，直至分划数为 0 。
2. 由于分划与顺序无关，为避免重复，按照从大到小依次设定每一个分划值，即前面的值 $\geq$ 后面的值。
3. 将确定的分划值存入数组中



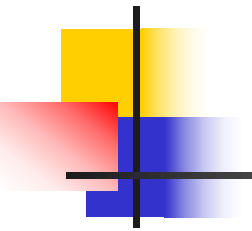
递归求解整数的分划问题

```
void split(int n, int cur ) // 将n从位置cur分划为一系列整数和
{
    if( !n)      /* 递归边界： 已确定一种分划 */
        输出该分划方案
    else
        for(i = n; i >=1; i--) /* 枚举 当前位置cur的分划值 */
        {
            if( i值可行) {
                a[cur]=i;      // 取 i 作为当前位置的分划值
                split(n-i,cur+1); /* 递归确定下一位置的分划值*/
            }
        }
}
```

```

int a[100]={0};
void split(int n,int cur)
{
    int i;
    if(n==0) out(cur);
    else
        for(i=n;i>=1;i--)/*依次选n至1作为当前位置的值*/
            if(cur==0||i<=a[cur-1]) {/*当前需要确定的是首元素,
                                     或者待选值不大于上一位置的值 */
                a[cur]=i;/*选i*/
                split(n-i,cur+1);/*递归确定下一位置的值*/
            }
}

```



枚举的递归实现

用递归实现枚举

赛软件 * 比赛 = 软件比拼

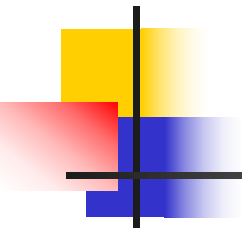
a b c d a b c d e

int a[5]; // a[0], a[1], a[2], a[3], a[4]

从 k=0 开始 **穷举**a[k], 直至 k=5 结束

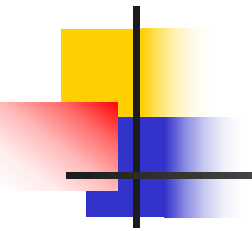
for(i=0; i<10; i++)

if(i未用过) a[k]=i;



枚举的递归实现

```
void find(int a[ ],int k)
{ int i,j,count;
  if(k==5) /* 递归边界：已枚举了5个汉字 */
  {   算式成立，则输出   }
  else
    for(i = 0; i < 10; i++) { /* 枚举a[k] */
      判断 i 在a[0]~a[k-1]中是否用过
      if(i未用过) {
        a[k]=i;
        find(a,k+1); /* 枚举a[k+1] */
      }
    }
}
```



递归练习

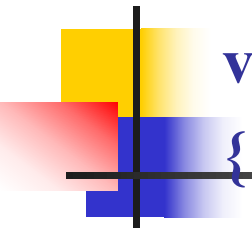
八皇后问题：在8X8格的棋盘上摆放8个皇后，使它们互不攻击，即任意2个皇后不能在同一行或同一列或同一条对角线上。找出所有解。

`int a[8];` 记录可行解

`a[i]`表示第*i*行皇后放在第`a[i]`列

`a={0,5,7,2,6,3,1,4}`

从 `k=0` 开始递归穷举`a[k]`，直至 `k=8` 结束



```
void find(int a[ ],int i )
```

```
{
```

```
    if(i==8) /* 递归边界：已枚举每一行 */
```

```
        {  输出  }
```

```
    else
```

```
        for(j = 0; j <8; j++) /* 将i行皇后放到 j 列 */
```

```
        {
```

```
            判断 是否互不攻击
```

```
            if( 互不攻击 ) {
```

```
                a[i]=j;
```

```
                find(a,i+1); /* 枚举a[i+1] */
```

```
            }
```

```
        }
```